

Confluence Test Agent

Runs your tests straight from Confluence — no code needed. It writes the test scripts itself, fixes them automatically when the website changes, and can be set up for a new project in about an afternoon.

BUILT ON

Playwright · Cucumber · Confluence · Claude

STATUS

Working on Hyundai Australia — proposing wider reuse

AUDIENCE

Devs & QA leadership

The usual way of testing breaks down fast.

This is how automated testing normally works — before a framework like this exists.

- Only a developer can write or update a test. QA has an idea for a test case but has to wait in line for dev time.

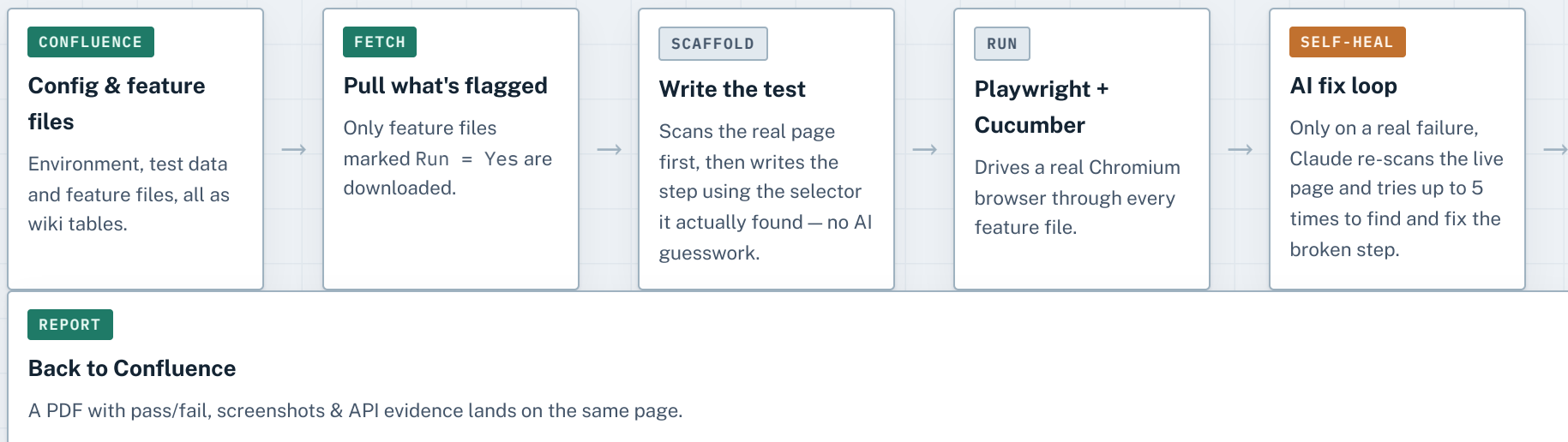
- The smallest website change — even one nobody would call a "change," like a button's underlying code being swapped — breaks tests overnight. Not because the product is broken, but because the test was too fragile to notice the difference.

- Test results live in a technical dashboard that only engineers ever open — so QA leads and managers rarely see whether things are actually passing.

- Adding one new feature file means changing code and waiting for a developer to review and approve it — for something that's really just "test this form too."

One pipeline, driven entirely from Confluence.

Test cases, test data, and environment config live in Confluence tables — not in code. One command runs the whole loop.



One command closes the whole loop: `node scripts/agentOrchestrator.js`

Locking a script costs nothing. Once a step file works the way you want, mark it `// @locked` — every future run skips it completely, so Claude is never called and no tokens are spent on it. Claude only switches on when a test genuinely fails, and even then it's capped at 5 attempts, each one re-checking the live page like the self-heal step above.

Plain English in. A working test out.

This is a complete, real feature file — nothing below it is hand-coded by a developer.



Feature: Hyundai Book a Test Drive flow

Background:

Given the user has loaded the test data from the Confluence page

And the user navigates to Test drive

Scenario: User submits a test drive booking

Given the user has selected a vehicle, powertrain and dealer

When the user enters a valid first name

And the user enters a valid email address

And the user enters a valid phone number

And the user accepts the privacy consent checkbox

And the user submits the test drive booking

Then the booking should be submitted successfully

You don't have to write even this by hand. Describe the feature file in plain English and ask Claude to draft the Gherkin for you — QA reviews it, ticks Run = Yes, and the agent does the rest.

Does the price on the calculator match the price in our system?

A real check the agent runs today: it logs into the internal pricing admin, reads a vehicle's Manufacturer List Price and Drive-Away price, opens the public Car Price Calculator, matches the same vehicle by its FCAI code, and confirms the two agree — to the cent — across every model and variant.

TC#	FEATURE FILE	STATUS	SCREENSHOT	API PAYLOAD	SUMMARY
04	KONA — Drive-Away price match	PASS	captured	200	Admin: \$34,500 · Calculator: \$34,500
05	TUCSON — Contact a dealer submission	FAIL	captured	200	the Powertrain dropdown has no selectable options — required field could not be filled

Every row carries a screenshot and the raw API response as evidence, so anyone can audit *why* it passed. And when it fails, the summary names the exact field or condition at fault — never a bare "failed."

Built for Hyundai. Not built to stay Hyundai-only.

TODAY

- One repository. The generic engine — fetch, generate, self-heal, report — sits next to Hyundai's feature files and step definitions.
- Starting a new project means cloning the whole repo, then manually deleting everything Hyundai-specific.

PROPOSED

- Split the repo: a **core engine** (project-agnostic) and a thin **project config** layer (your Confluence page IDs, your feature files).
- A new project starts from the core template with nothing to delete — just point it at your own Confluence page.

This already works, minus the packaging. We tested exactly this path — pointed the framework at a different codebase and asked Claude to strip out everything Hyundai-specific. It produced a working base for a new project in under an hour. Packaging a clean starter template removes even that step.

No developer required for day-to-day use.

ONE-TIME SETUP

1. Install Node.js, Git, and VS Code.
2. `npm install` and `npx playwright install`.
3. Add your own Confluence email + API token to `.env`.

EVERY DAY

1. Open the Confluence Feature Files table.
2. Add or edit a test-data row, tick Run = Yes.
3. Run one command; read the PDF report on the same page.

New feature file? Describe it in plain English and ask Claude to draft the Gherkin — no selectors, no Playwright code, ever, for QA to hand-write.

Four things worth remembering

01

Not BAU-specific

It isn't built for one fixed set of tests. Point it at any feature or page, describe the flow in plain English, and the same engine builds and runs it — already proven on a different codebase in under an hour.

02

Driven entirely by Confluence

Test data, environment config, and which feature files run all live in wiki tables — QA changes them with no PR.

03

Self-healing

Routine page changes are absorbed automatically; a genuine failure comes back with a named cause, not just "failed."

04

Caches what it already learned

DOM maps, generated step code, and healed selectors persist between runs — re-testing a feature doesn't start from zero.

FOR DISCUSSION

1. Approve packaging the core engine as a reusable starter template.
2. Nominate the next project to pilot it on.
3. Confirm rollout & training time for the QA team.